

Alembic — Database Migrations for SQLAlchemy

Practice project: setting up a migration environment against a PostgreSQL database running in Docker, writing migrations by hand, then auto-generating them from SQLAlchemy models.

Project files

File	Purpose
<code>alembic.ini</code>	Alembic config — script location, database URL, logging
<code>models.py</code>	SQLAlchemy models (the <code>Company</code> model + <code>Base</code>)
<code>myapp/env.py</code>	Runs on every migration command; connects to the DB and wires up <code>target_metadata</code>
<code>myapp/script.py.mako</code>	Template used to generate every new migration file
<code>myapp/README</code>	One-line description of the environment type
<code>myapp/versions/</code>	The actual migration scripts
<code>requirements.txt</code>	<code>alembic</code> , <code>psycopg2</code>

Table of Contents

1. [Alembic](#)
2. [Installation](#)
3. [Migration Environment](#)
4. [Telling Alembic How to Connect to the Database](#)
5. [Let's Create Migrations](#)
6. [Running Migrations Against the Database](#)
7. [Downgrading in Alembic](#)
8. [Auto-Generate Migrations from SQLAlchemy Models](#)
9. [Command Reference](#)
10. [Full Reference Files](#)
11. [Things To Watch Out For](#)
12. [Appendix: Markdown Cheat Sheet](#)

1. Alembic

Alembic is a lightweight database migration tool for Python written by the creator of SQLAlchemy. It handles tracking, creating, and applying schema changes safely across different environments.

Why it matters: `Base.metadata.create_all()` only ever *creates* tables that don't exist. It cannot alter a column, rename a table, or roll anything back. Alembic gives every schema change a version number, an `upgrade()` and a `downgrade()`, so the database can move forwards and backwards in a controlled way.

2. Installation

Install `alembic` and `psycopg2` in the virtual environment.

Reference code — `requirements.txt` (complete file):

```
alembic
psycopg2
```

```
pip install -r requirements.txt
```

- `psycopg2` is the PostgreSQL driver Alembic/SQLAlchemy connect through.

3. Migration Environment

Usage of Alembic starts with the creation of the **Migration Environment**. This is a directory of scripts that is specific to a particular application. The migration environment is created just once, and is then maintained along with the application's source code itself. The environment is created using the `init` command of Alembic, and is then customizable to suit the specific needs of the application.

The structure of this environment, including some generated migration scripts, looks like:

```
yourproject/
  alembic.ini
  pyproject.toml
  alembic/
    env.py
    README
    script.py.mako
    versions/
      3512b954651e_add_account.py
      2b1ae634e5cd_add_order_id.py
      3adcc9a56557_rename_username_field.py
```

Run the following command to create a migration environment:

```
alembic init <Environment_name>
```

(In my practice I used `myapp` as the environment name — that is why `alembic.ini` has `script_location = %(here)s/myapp`.)

It creates the `alembic.ini` file, and this is where the Alembic scripts look when we invoke the Alembic commands.

We also get the `<environment_name>` folder with a `versions` folder (initially empty) and `env.py`.

What each generated file is

File	What it does
<code>alembic.ini</code>	Config file. Script location, DB URL, logging setup.
<code>env.py</code>	Python script run any time the Alembic migrations are invoked.
<code>README</code>	Just says which template was used.
<code>script.py.mako</code>	The template every new migration file is generated from.
<code>versions/</code>	Where the migration scripts land.

`env.py` is a Python script that is run any time the Alembic migrations are invoked. This file will always be running when we perform our migrations. It is going to use SQLAlchemy and its provided engine to connect to the database.

`README` (complete file):

```
Generic single-database configuration.
```

`script.py.mako` — this is the template. Every value in `${...}` gets filled in by Alembic when a new revision is created, which is why every migration file looks the same:

```
"""${message}

Revision ID: ${up_revision}
Revises: ${down_revision | comma,n}
Create Date: ${create_date}

"""
from typing import Sequence, Union

from alembic import op
import sqlalchemy as sa
${imports if imports else ""}

# revision identifiers, used by Alembic.
revision: str = ${repr(up_revision)}
down_revision: Union[str, Sequence[str], None] = ${repr(down_revision)}
branch_labels: Union[str, Sequence[str], None] = ${repr(branch_labels)}
depends_on: Union[str, Sequence[str], None] = ${repr(depends_on)}

def upgrade() -> None:
```

```

"""Upgrade schema."""
${upgrades if upgrades else "pass"}

def downgrade() -> None:
    """Downgrade schema."""
    ${downgrades if downgrades else "pass"}

```

4. Telling Alembic How to Connect to the Database

Now we are going to tell Alembic how to connect to our database in the Docker container.

- Go to the `alembic.ini` file and find `sqlalchemy.url`.
- `sqlalchemy.url = driver://user:pass@localhost/dbname`. We need a driver (postgres/MySQL etc.), we need the user and password, localhost, and the name of the database.
- So for this practice I replaced it with `postgresql://postgres:secret@localhost/alembic_db`, which I previously created.

Reference code — `alembic.ini`:

```

[alembic]
# path to migration scripts.
script_location = %(here)s/myapp

# sys.path path, will be prepended to sys.path if present.
prepend_sys_path = .

path_separator = os

# database URL. This is consumed by the user-maintained env.py script only.
sqlalchemy.url = postgresql://postgres:secret@localhost/alembic_db

```

Breaking down the URL

```

postgresql://postgres:secret@localhost/alembic_db
driver      user  password  host    database name

```

Note: `prepend_sys_path = .` is what lets `env.py` do `from models import Base` — it adds the project root to `sys.path` so my `models.py` is importable.

Note on logging: `alembic.ini` also holds the logging config (`[loggers]`, `[handlers]`, `[formatters]`). `logger_alembic` is set to `INFO`, which is why we see the "Running upgrade ..." lines in the terminal.

5. Let's Create Migrations

Run:

```
alembic revision -m "<message>"
```

It is going to create a migration inside `<environment_name>/versions/<hashid>.py`.

Inside a migration file

- Inside that migration version we can find `down_revision` set to `None`. It is set to `None` in only the first migration. The purpose of this variable is to point to the ID of the previous migration in the Alembic configuration.
- There will be two functions named `upgrade()` and `downgrade()`.
- In Alembic we can add code to both these functions to tell it what happens on upgrade and on downgrade.
- Later we can auto-generate the code that is added to these functions from SQLAlchemy models.
- We have imported `from alembic import op` and `import sqlalchemy as sa`. So we can use them for database operations inside our `upgrade()` and `downgrade()` functions in our migrations.

`upgrade()` example

```
def upgrade() -> None:
    op.create_table(
        "employee",
        sa.Column("id", sa.Integer, primary_key=True),
        sa.Column("name", sa.String(50), nullable=False),
        sa.Column("current", sa.Boolean, default=True)
    )
```

`downgrade()` example

```
def downgrade() -> None:
    op.drop_table("employee")
```

Rule of thumb: `downgrade()` should exactly undo `upgrade()`. If `upgrade()` creates a table, `downgrade()` drops it. If `upgrade()` adds a column, `downgrade()` drops that column.

6. Running Migrations Against the Database

```
alembic upgrade <revision_id> # upgrade to a specific migration
```

```
alembic upgrade head # upgrade to the most recent migration
```

The revision ID is found in the migration file if we want to upgrade to a specific migration; `head` can be used for the most recent migration. Now the `upgrade()` function will execute.

Checking the database

We can see the database using the `docker exec` command:

```
docker exec -ti <containerName> <connectUsing(ex. psql)> <user> -d <databaseName>
```

And then:

Command	What it does
<code>\dt</code>	Show all tables
<code>select * from employees;</code>	Any normal SQL query
<code>\q</code>	Quit

Adding another revision

We can add another revision/migration using:

```
alembic revision -m "<message>"
```

Now in this version we have `down_revision: '6909a6a9797e'`, which points to the previous revision. This was `None` in the first migration. This is used to keep track of the historical data of revisions. We can again define the `upgrade()` and `downgrade()` functions to use them, and we can also make other migrations after this.

So the revisions form a linked chain:

```
None —> 6909a6a9797e —> 9da62ec54f38 —> ... —> head
      (first)           (add job_title)
```

Now when we run the command `alembic upgrade head`, it is going to apply what was coded in the `upgrade()` function.

Viewing history

We can also use the `alembic history` command to be provided with the upgrade history in the terminal:

```
Running upgrade 6909a6a9797e -> 9da62ec54f38, add job_title column
```

We can track all these changes in our **pgAdmin 4** by connecting our Postgres container database to a server in pgAdmin 4.

Note: Alembic records the current revision in a table it creates itself, called `alembic_version`. That single row is how it knows where the database currently stands. You will see it in `\dt` alongside your own tables.

7. Downgrading in Alembic

To downgrade we can apply a relative migration.

Relative Migration

Relative migrations operate in place from the **current** migration.

To get the current migration:

```
alembic current
```

- To relatively downgrade we can use the command `alembic downgrade <n>`, where `n` is a **negative** number, and it is going to downgrade relatively from the current migration.
- We can use `alembic upgrade <n>`, where `n` is a **positive** number, to upgrade relatively.

⚠ Important: the sign must actually be written. It is `alembic downgrade -1` and `alembic upgrade +2`. A bare `alembic upgrade 2` is read as a revision ID, not a relative step.

- To completely downgrade we can use the `alembic downgrade base` command. That is going to completely reverse everything and go back to its original state.
- To completely upgrade we can use the `alembic upgrade head` command. That is going to completely upgrade everything and go back to its upgraded state.

```
base <----- downgrade ----- current ----- upgrade -----> head
```

8. Auto-Generate Migrations from SQLAlchemy Models

Writing migrations by hand is tedious and easy to get wrong — a forgotten index or a mistyped column type only surfaces once it hits production. Alembic solves this with **autogeneration**: it inspects your SQLAlchemy models, compares them to the live database schema, and produces a migration script containing exactly the differences. You review the result, adjust anything it couldn't infer, and commit it alongside your model changes.

This works because Alembic reads the same `MetaData` object your models are registered on. Once `target_metadata` is wired up in `env.py`, a single `alembic revision --autogenerate` is enough to keep schema and code in sync. This places so-called **candidate migrations** into our new migrations file. We review and modify these by hand as needed, then proceed normally.

Note: Autogenerate detects table and column additions/removals and most type changes, but it won't catch things like column renames (it sees a drop plus an add), server-side default changes, or constraint tweaks on some backends. **The generated script is a draft, not a final answer.**

Wiring up `target_metadata`

To use autogenerate, we first need to modify our `env.py` so that it gets access to a table metadata object that contains the target.

Suppose our application has a declarative base in `myapp.mymodel`. This base contains a `MetaData` object which contains `Table` objects defining our database. We make sure this is loaded in `env.py` and then passed to `EnvironmentContext.configure()` via the `target_metadata` argument. The `env.py` sample script used in the generic template already has a variable declaration near the top for our convenience, where we replace `None` with our `MetaData`.

Replace in `env.py`:

From:

```
# add your model's MetaData object here
# for 'autogenerate' support
# from myapp import mymodel
# target_metadata = mymodel.Base.metadata
target_metadata = None
```

To (this is the docs' generic example):

```
from myapp.mymodel import Base
target_metadata = Base.metadata
```

What I actually did, since my `models.py` sits at the project root:

```
from models import Base

target_metadata = Base.metadata
```

The models being tracked

Reference code — `models.py` (complete file):


```

from sqlalchemy import Column, DateTime, String, Integer, func
from sqlalchemy.ext.declarative import declarative_base

Base = declarative_base()
metadata = Base.metadata

class Company(Base):
    __tablename__ = 'company'

    id = Column(Integer, primary_key=True)
    name = Column(String(60), unique=True)
    address = Column(String(100), nullable=True)
    created_at = Column(DateTime, default=func.now())

    def __repr__(self):
        return f"id: {self.id}, name: {self.name}"

```

- `unique=True` on `name` becomes a `UniqueConstraint` in the migration.
- `default=func.now()` sets the timestamp at insert time.
- `__repr__` is only for printing the object in Python — it has no effect on the schema.

Running autogenerate

Then we can use the command:

```
alembic revision --autogenerate -m "<message>"
```

Migration 1 — Added Company model

Reference code — `myapp/versions/b79af0eb0b9b_added_company_model.py`:

```

"""Added Company model

Revision ID: b79af0eb0b9b
Revises:
Create Date: 2026-08-03 16:39:01.161188

"""
from typing import Sequence, Union

from alembic import op
import sqlalchemy as sa

# revision identifiers, used by Alembic.
revision: str = 'b79af0eb0b9b'
down_revision: Union[str, Sequence[str], None] = None

```

```

branch_labels: Union[str, Sequence[str], None] = None
depends_on: Union[str, Sequence[str], None] = None

def upgrade() -> None:
    """Upgrade schema."""
    # ### commands auto generated by Alembic - please adjust! ###
    op.create_table('company',
        sa.Column('id', sa.Integer(), nullable=False),
        sa.Column('name', sa.String(length=60), nullable=True),
        sa.Column('created_at', sa.DateTime(), nullable=True),
        sa.PrimaryKeyConstraint('id'),
        sa.UniqueConstraint('name')
    )
    op.drop_table('employee')
    # ### end Alembic commands ###

def downgrade() -> None:
    """Downgrade schema."""
    # ### commands auto generated by Alembic - please adjust! ###
    op.create_table('employee',
        sa.Column('id', sa.INTEGER(), autoincrement=True, nullable=False),
        sa.Column('name', sa.VARCHAR(length=50), autoincrement=False, nullable=False),
        sa.Column('current', sa.BOOLEAN(), autoincrement=False, nullable=True),
        sa.Column('job_title', sa.VARCHAR(length=64), autoincrement=False,
        nullable=True),
        sa.PrimaryKeyConstraint('id', name=op.f('employee_pkey'))
    )
    op.drop_table('company')
    # ### end Alembic commands ###

```

Two things to notice here — this is the whole lesson about autogenerate:

1. It generated `op.drop_table('employee')` on its own. The `employee` table (from my earlier hand-written migrations) still existed in the live database, but there is no `Employee` class in `models.py`. Autogenerate compares **models vs. live database**, saw a table with no model, and decided to drop it. That is why the header says *"please adjust!"* — if that had been a real table with real data, running this would have destroyed it.
2. It wrote a matching `downgrade()` that recreates `employee` with all its columns, including `job_title` which I had added in an earlier migration. Alembic read that shape from the live database, not from my code.

Also worth noting: `down_revision` is `None` here even though I had made hand-written migrations before, because this was a fresh start of the `versions/` chain.

Migration 2 — Added Address

I then added `address = Column(String(100), nullable=True)` to the `Company` model and ran autogenerate again.

Reference code — `myapp/versions/bc563bd30aed_added_address.py`:

```

"""Added Address

Revision ID: bc563bd30aed
Revises: b79af0eb0b9b
Create Date: 2026-08-03 16:40:29.064582

"""
from typing import Sequence, Union

from alembic import op
import sqlalchemy as sa

# revision identifiers, used by Alembic.
revision: str = 'bc563bd30aed'
down_revision: Union[str, Sequence[str], None] = 'b79af0eb0b9b'
branch_labels: Union[str, Sequence[str], None] = None
depends_on: Union[str, Sequence[str], None] = None

def upgrade() -> None:
    """Upgrade schema."""
    # ### commands auto generated by Alembic - please adjust! ###
    op.add_column('company', sa.Column('address', sa.String(length=100),
    nullable=True))
    # ### end Alembic commands ###

def downgrade() -> None:
    """Downgrade schema."""
    # ### commands auto generated by Alembic - please adjust! ###
    op.drop_column('company', 'address')
    # ### end Alembic commands ###

```

This is autogenerate at its best — one column added to the model, one clean `add_column` / `drop_column` pair generated. Note `down_revision` now points at `'b79af0eb0b9b'`, chaining the two migrations together.

The full chain so far:

```

None —> b79af0eb0b9b —> bc563bd30aed
      (Company)         (Address)      = head

```

9. Command Reference

Command	What it does
<code>alembic init <env_name></code>	Create the migration environment (once per project)
<code>alembic revision -m "msg"</code>	Create an empty migration to fill in by hand
<code>alembic revision --autogenerate -m "msg"</code>	Create a migration by diffing models against the DB
<code>alembic upgrade head</code>	Apply everything up to the latest
<code>alembic upgrade <revision_id></code>	Apply up to a specific revision
<code>alembic upgrade +2</code>	Move forward 2 revisions from current
<code>alembic downgrade -1</code>	Move back 1 revision from current
<code>alembic downgrade base</code>	Undo everything, back to an empty schema
<code>alembic current</code>	Show which revision the DB is on right now
<code>alembic history</code>	Show the full chain of revisions

10. Full Reference Files

requirements.txt

```
alembic
psycopg2
```

models.py

```
from sqlalchemy import Column, DateTime, String, Integer, func
from sqlalchemy.ext.declarative import declarative_base

Base = declarative_base()
metadata = Base.metadata

class Company(Base):
    __tablename__ = 'company'

    id = Column(Integer, primary_key=True)
    name = Column(String(60), unique=True)
    address = Column(String(100), nullable=True)
    created_at = Column(DateTime, default=func.now())

    def __repr__(self):
        return f"id: {self.id}, name: {self.name}"
```

myapp/env.py

```
from logging.config import fileConfig

from sqlalchemy import engine_from_config
from sqlalchemy import pool

from alembic import context
from models import Base

# this is the Alembic Config object, which provides
# access to the values within the .ini file in use.
config = context.config

# Interpret the config file for Python logging.
# This line sets up loggers basically.
if config.config_file_name is not None:
    fileConfig(config.config_file_name)

# add your model's MetaData object here
# for 'autogenerate' support
# from myapp import mymodel
# target_metadata = mymodel.Base.metadata
target_metadata = Base.metadata

# other values from the config, defined by the needs of env.py,
# can be acquired:
# my_important_option = config.get_main_option("my_important_option")
# ... etc.

def run_migrations_offline() -> None:
    """Run migrations in 'offline' mode.

    This configures the context with just a URL
    and not an Engine, though an Engine is acceptable
    here as well.  By skipping the Engine creation
    we don't even need a DBAPI to be available.

    Calls to context.execute() here emit the given string to the
    script output.

    """
    url = config.get_main_option("sqlalchemy.url")
    context.configure(
        url=url,
        target_metadata=target_metadata,
        literal_binds=True,
        dialect_opts={"paramstyle": "named"},
    )

    with context.begin_transaction():
        context.run_migrations()
```

```
def run_migrations_online() -> None:
    """Run migrations in 'online' mode.

    In this scenario we need to create an Engine
    and associate a connection with the context.

    """
    connectable = engine_from_config(
        config.get_section(config.config_ini_section, {}),
        prefix="sqlalchemy.",
        poolclass=pool.NullPool,
    )

    with connectable.connect() as connection:
        context.configure(
            connection=connection, target_metadata=target_metadata
        )

        with context.begin_transaction():
            context.run_migrations()

if context.is_offline_mode():
    run_migrations_offline()
else:
    run_migrations_online()
```

Offline vs. online mode, in plain terms:

- **Online** — the normal case. Alembic builds an engine, opens a real connection, and runs the SQL against the database.
- **Offline** (`alembic upgrade head --sql`) — no connection at all. Alembic just *prints* the SQL it would have run, so a DBA can review it or run it manually in production.

alembic.ini (the parts I actually touched)

```
[alembic]
script_location = %(here)s/myapp
prepend_sys_path = .
path_separator = os

sqlalchemy.url = postgresql://postgres:secret@localhost/alembic_db

[loggers]
keys = root,sqlalchemy,alembic

[logger_alembic]
level = INFO
```

```
handlers =
qualname = alembic
```

myapp/README

Generic single-database configuration.

11. Things To Watch Out For

1. **Always read an autogenerated migration before running it.** Mine silently generated `op.drop_table('employee')`. On a real database that is data loss. The `# please adjust!` comment is not decoration.
2. **Autogenerate cannot see a rename.** Rename a column and it will produce a drop plus an add — which throws away the data in that column. Rewrite it by hand as `op.alter_column(..., new_column_name=...)`.
3. **Relative migrations need the sign.** `alembic downgrade -1`, `alembic upgrade +2`. Without the sign it is treated as a revision ID.
4. **The database password is sitting in `alembic.ini` in plain text.** Fine for practice; a real project reads it from an environment variable inside `env.py` instead, using `config.set_main_option("sqlalchemy.url", os.getenv("DATABASE_URL"))`.
5. **`sqlalchemy.ext.declarative.declarative_base` is the legacy import path.** It still works but is deprecated in SQLAlchemy 2.0 — the current one is `from sqlalchemy.orm import declarative_base`.
6. **Pin your versions.** `requirements.txt` currently has bare `alembic` and `psycopg2`. On Windows, `psycopg2` often fails to build — `psycopg2-binary` is the prebuilt drop-in.
7. **Never edit a migration that has already been applied and shared.** Create a new one instead. Editing history breaks everyone else's `alembic_version` row.

12. Appendix: Markdown Cheat Sheet

Since I wasn't sure how I made the headings — this is how Markdown works:

What I want	What I type
Big heading	<code># Heading 1</code>
Sub heading	<code>## Heading 2</code>
Smaller sub heading	<code>### Heading 3</code>
Bold	<code>**bold**</code>

What I want	What I type
<i>Italic</i>	<code>*italic*</code>
<code>inline code</code>	<code>`code`</code>
Bullet list	<code>- item</code>
Numbered list	<code>1. item</code>
Quote / note box	<code>> note</code>
Horizontal line	<code>---</code>
Link	<code>[text](https://url)</code>

What went wrong in my original notes: I used `##` for the top title "Alembic" and then `#` for every section under it. That's backwards — `#` is the *biggest* heading, `##` is a level down. Headings should get *smaller* as you go deeper, never bigger.

Code block — three backticks, then the language name, then the code, then three backticks again:

```
```python
print("hello")
```
```

The language name (`python`, `bash`, `ini`, `text`, `sql`) is what gives the code its colours.

Table — pipes for columns, dashes for the header line:

```
Column A	Column B
value 1	value 2
```

Tip: In VS Code press `Ctrl + Shift + V` to preview a `.md` file and see it rendered.